

Riassunto del lavoro — stato attuale

Progetto Invisible Engine · adattamento Stake Engine ↔ Play4Fun

Cosa ho costruito finora

- Un **traduttore** (`packages/rgs-translator-eagaming`) che fa da ponte tra il protocollo Play4Fun (usato da EAGaming / Hot Fruits) e quello atteso dallo Stake Engine.
- Un **mock server** locale (`scripts/mock-rgs-server.mjs`) che imita Play4Fun, così posso sviluppare e testare senza dipendere dal server reale.
- Un **facade** che espone l'intero sistema nella forma che lo Stake Engine già conosce (stessi nomi di funzioni, stessa shape dei dati).
- L'app `lines` punta al facade tramite alias `Vite` — nessuna modifica all'engine originale.

Cosa ho scoperto sul protocollo Play4Fun

- Endpoint unico `/rgs/engine` con azioni `bet`, `play`, `collect`.
- Eventi osservati: `bet`, `gameStart`, `spinStart`, `spinWin`, `playedSpin`, `gameEnd`, `gameRoundOver`.
- Simboli osservati: `PIC1-PIC7` + `SCAT` (mappati a `L1-L5`, `H1-H2`, `S` di Stake).
- **Importi: interi in centesimi** ($100 = \$1.00$). Stake usa interi in milionesimi ($1.000.000 = \$1.00$). Fattore di conversione: `10.000`. Già gestito nel facade.
- Due modalità di round: manuale (`collect` separato) e auto-`collect`.

Cosa funziona

- Smoke test del mock: **OK**
- Smoke test del facade: **OK**
- Flusso completo `bet` → `play` → `collect` con saldo aggiornato correttamente: **OK**
- Phase 2 chiusa, tutto pushato su `origin/main`.

Cosa so dello Stake Engine (architettura)

- Monorepo `Turborepo` con **6 app di gioco** (`lines`, `cluster`, `scatter`, `ways`, `number-picker`, `price`) e ~30 pacchetti condivisi.
- Stack: **Svelte 5** + **SvelteKit**, **PixiJS 8** per il rendering, **XState 5** per la macchina a stati del gioco, **pixi-svelte** come ponte dichiarativo interno.
- Ogni app è autosufficiente: stessa struttura ma config, simboli e regole indipendenti.
- Il protocollo di comunicazione con il server è isolato in `packages/rgs-requests` — è il punto in cui si innesta il mio facade senza toccare le app.

Cosa ho scoperto oggi sui config file

- Ogni gioco ha **un solo file** `apps/<gioco>/src/game/config.ts` che contiene tutto: simboli, paylines, payable, layout dei rulli (basegame e freegame), modalità di scommessa, RTP, max win.
- Il file viene **importato staticamente** a build-time. Il server non manda mai un config: invia solo eventi che fanno riferimento a simboli/posizioni che il config dovrebbe già conoscere.
- I tipi TypeScript sono **derivati dal config** (`type SymbolName = keyof typeof config.symbols`): cambio il config e il sistema dei tipi si aggiorna automaticamente. Però questa protezione vale solo a compile-time — i dati che arrivano dal server la bypassano.
- **Sostituire un config è facile in teoria** (un solo file), ma le dimensioni della griglia sono ancora hard-coded in `constants.ts`. Per portare davvero un'app su un altro gioco serve toccare anche quel file.
- **Robustezza: zero tolleranza ai mismatch.** Se il server manda un simbolo che il config non conosce, o una posizione fuori dalla griglia, l'app **crasha rumorosamente** (`TypeError` sull'accesso all'array). Nessun fallback, nessun simbolo di default, nessun controllo dei limiti. Lo dico come constatazione tecnica, non come critica: è un trade-off ragionevole quando server e client sono dello stesso fornitore, ma diventa un punto critico quando si fa adattamento tra protocolli diversi come stiamo facendo noi.

Aggiornamento — il config arriva dal server

Scoperta importante: Play4Fun **invia il proprio config come primo evento** della sessione (`event: 'config'`). Non serve scrapare il bundle del client: il server stesso dichiara simboli, griglia, paylines e payable.

- **Vocabolario simboli chiuso:** 8 voci (PIC1–PIC7 + SCAT). Nessuna sorpresa possibile a runtime.
- **Niente wild** (`wildSymbols: []`) e niente freegame dichiarato.
- **Griglia 5×3** confermata (`window: {reels:5, rows:3}`).
- **5 paylines, non 20** — correzione rispetto a quanto avevamo stimato. I 5 pattern di Hot Fruits coincidono con i primi 5 pattern del config Stake Lines (in ordine diverso).
- **Paytable invertita rispetto alla nostra mappatura iniziale:** PIC1 è il simbolo che paga di più (5-of-a-kind = 5000), PIC7 il meno pagante (paga persino 2-of-a-kind = 5). La mappa simboli del traduttore va corretta.
- **Verifica matematica:** uno spinWin reale catturato in passato ($PIC4 \times 3 = \text{pay } 40$ con `betPerLine 2`) coincide con la payable ($PIC4 \rightarrow 20 \times 2 = 40$). Conversioni di scala corrette.

Implicazioni e prossimi passi da discutere

- 1 **Difesa al confine del facade.** Ora che il server dichiara il config, il facade può costruire al boot un whitelist di simboli, un clamp della griglia e una paytable di confronto, validando ogni evento successivo contro questa fonte di verità. Niente più crash silenziosi su simbolo sconosciuto.
- 2 **Cross-check Stake ↔ Play4Fun.** Al boot il facade può confrontare il config statico di Stake con quello del server e segnalare ogni mismatch (es. "Stake si aspetta W ma Play4Fun non manda mai wild").
- 3 **Correzione mappa simboli.** Da rivedere insieme: allineare per *payout assoluto* (PIC1→H1) o per *rank simbolico*. Decisione di design.
- 4 **Mismatch semantici residui** — quando Stake si aspetta campi che Play4Fun non invia. Tre strategie possibili: sintetizzare un default sensato, rendere il campo opzionale nell'engine, oppure fallire esplicitamente al confine. Da decidere caso per caso.